EST-744

Embeddings y Aprendizaje de Representaciones

Juan Zamora O.

Deep Learning para Estadísticos





El Problema: El Texto es Simbólico

¿Cómo alimentamos palabras a redes neuronales?

El desafío fundamental es que las palabras son símbolos discretos sin estructura numérica inherente. Las redes neuronales operan sobre números y geometrías continuas. Necesitamos construir un puente.

Enfoques tradicionales (insuficientes):

- **Codificación one-hot**: “gato” → [0, 0, ..., 1, 0, ..., 0]
  - Vocabulario de 10,000 palabras = 10,000 dimensiones
  - No captura relaciones semánticas (todas las palabras equidistantes)
  - Vectores enormes y dispersos (99.99% ceros)
  - Desperdicia la estructura subyacente del lenguaje

Insight fundamental: Aprender representaciones densas donde el significado se codifica en la **geometría** del espacio de vectores, no en la presencia/ausencia de dimensiones

La Idea Central

Un **embedding** es un mapeo aprendido que transforma cada palabra en un vector denso de dimensionalidad mucho menor (típicamente 50-300).

```
"gato"   → [0.23, -0.15, 0.08, ..., 0.42]    (300 dimensiones)
"perro"  → [0.25, -0.13, 0.07, ..., 0.41]
"rey"    → [0.12,  0.45, -0.22, ..., 0.61]
"reina"  → [0.10,  0.50, -0.20, ..., 0.58]
```

Propiedad emergente: Palabras similares tienen vectores cercanos en el espacio

¿Por qué funciona? La distancia euclidiana entre vectores codifica similitud semántica:

$$\text{similaridad} \propto \frac{1}{1 + \text{distancia}}$$

La Hipótesis Distribucional

“Conocerás una palabra por la compañía que frecuenta” (Firth, 1957)

Principio linguístico: Palabras que aparecen en contextos similares tienen significados similares

Ejemplo práctico: - “gato” aparece con: “mascota”, “ronroneo”, “felino”, “suave” - “perro” aparece con: “mascota”, “ladrido”, “canino”, “leal” - **Contexto compartido** → los vectores deberían ser cercanos

Este principio es el fundamento teórico de todos los métodos modernos de embeddings.

Idea Central del Modelo Skip-Gram

En lugar de tratar de predecir la siguiente palabra (como en un modelo de lenguaje), **predecimos las palabras contextuales a partir de la palabra objetivo**.

```
Oración: "Adoro mi [gato] muy tierno"
         Contexto: "Adoro", "mi", "muy", "tierno"
         Objetivo: "gato"

Tarea: Dado "gato" → predecir palabras contextuales
```

Arquitectura de la red: 1. Entrada: palabra objetivo codificada one-hot 2. Capa oculta: matriz de embeddings $W \in \mathbb{R}^{|V| \times d}$ (|V| = vocab, d = dim embedding) 3. Salida: distribución de probabilidad sobre palabras contextuales

Truco elegante: Después del entrenamiento, descartamos la capa de salida y usamos los **pesos de la capa oculta como embeddings**.

Proceso de Entrenamiento

1. **Extracción de pares**: Recorrer el corpus con ventana de contexto (ej: 5 palabras a cada lado)
2. **Construcción de ejemplos**: Para cada palabra objetivo, generar pares (objetivo, contexto)
3. **Optimización**: Maximizar probabilidad de palabras contextuales dada la palabra objetivo

$$\mathcal{L} = -\sum_{t=1}^{T} \sum_{-m \leq j \leq m, j \neq 0} \log P(w_{t+j}|w_t)$$

4. **Emergencia de estructura**: La red aprende automáticamente que palabras similares tienen vectores similares
5. **Resultado**: Vectores que capturan relaciones semánticas y sintácticas

Operaciones Vectoriales Significativas

Los embeddings capturan no solo similitud, sino **relaciones funcionales**:

$$\text{rey} - \text{hombre} + \text{mujer} \approx \text{reina}$$

¿Por qué funciona?

Piensa en cada dimensión del embedding como codificar un atributo semántico (no interpretable directamente, pero existe):

- “rey” tiene componentes de “autoridad” + “masculinidad”
- “hombre” codifica principalmente “masculinidad”
- Restar “hombre” elimina el componente de masculinidad
- Sumar “mujer” añade “feminidad”
- Resultado: “autoridad” + “feminidad” ≈ “reina”

Otras relaciones capturadas: - París - Francia + Londres ≈ Inglaterra (relaciones geográficas) - malo - bueno ≈ triste - feliz (analogías de sentimiento) - programador - programacion + música ≈ músico (roles vs. dominios)

Comparación de Alternativas

No se entrena embeddings desde cero; se usan modelos pre-entrenados en corpus masivos:

Método	Dimensiones	Corpus	Ventajas	Limitaciones
Word2Vec	300	Google News (3B palabras)	Rápido, intuitivo, eficiente	Palabras OOV desconocidas
GloVe	300	Wikipedia + Gigaword (6B)	Equilibra contexto local y global	Más lento en entrenamiento
FastText	300	Common Crawl (840B)	Maneja OOV via subpalabras	Archivos grandes (~7 GB)

¿Cuál elegir?

Integrando Embeddings en Modelos Deep Learning

En lugar de mandar one-hot encoding a la red, usamos una **capa de embeddings** que actúa como tabla de búsqueda:

```
keras.layers.Embedding(
    input_dim=vocab_size,       # ¿Cuántas palabras en vocabulario?
    output_dim=embedding_dim,    # Dimensionalidad del embedding (ej: 128)
    input_length=max_length      # Largo máximo de secuencia
)
```

Flujo de datos:

1. **Tokenización**: “el gato duerme” → [3, 42, 157]
2. **Embedding layer**: [3, 42, 157] → matriz de shape (3, embedding_dim)
   - Fila 0: embedding de palabra con índice 3
   - Fila 1: embedding de palabra con índice 42
   - Fila 2: embedding de palabra con índice 157
3. **Propagación**: Esta matriz se pasa a capas posteriores (LSTM, CNN, etc.)

Dos estrategias:

Del Nivel de Palabra al Documento

Los embeddings word-level son poderosos, pero ¿cómo representamos documentos completos?

Opción 1: Promediado Simple

```
oración = "el gato es muy feliz"
embedding_documento = mean([emb(el), emb(gato), emb(es), emb(muy), emb(feliz)]
```

Pros: Simple, rápido, funciona sorprendentemente bien **Contras**: Pierde orden de palabras (“no bueno” = “bueno no”)

Opción 2: Promediado Ponderado (TF-IDF)

$$\text{emb}_{\text{doc}} = \frac{\sum_i \text{tfidf}_i \cdot \text{emb}_i}{\sum_i \text{tfidf}_i}$$

Palabras comunes (artículos) contribuyen menos; palabras discriminativas contribuyen más.

Pros: Mitiga el impacto del ruido **Contras**: Aún pierde información de orden

Descripción de la Tarea

Objetivo: Predecir si una reseña de película es positiva o negativa

Dataset: IMDb (25,000 reseñas para entrenamiento, 25,000 para prueba)

Pipeline propuesto: 1. Cargar embeddings pre-entrenados (Word2Vec) 2. Tokenizar y convertir reseñas a secuencias de índices 3. Agregar embeddings (promedio simple) 4. Alimentar a clasificador simple (capas Dense)

Expectativa: ~85-90% de exactitud (limitado por promediado)

Cargando Embeddings Pre-entrenados

```python
from gensim.models import KeyedVectors
import numpy as np

# Cargar Word2Vec pre-entrenado (Google News)
# Nota: Descarga requiere ~1.5 GB
w2v = KeyedVectors.load_word2vec_format(
    'GoogleNews-vectors-negative300.bin', binary=True
)

# Verificar que funcionó
print(f"Vocabulario: {len(w2v)}")  # ~3 millones palabras
print(f"Dimensionalidad: {w2v.vector_size}")  # 300

# Probar similitud
print(w2v.similarity('gato', 'perro'))  # ~0.8 (muy similar)
print(w2v.similarity('gato', 'coche'))  # ~0.1 (poco similar)
```

Construyendo Matriz de Embeddings para Nuestro Vocabulario

```
# Crear matriz de embeddings para nuestro vocabulario específico
vocab_size = 10000
embedding_dim = 300
word_index = {}  # palabra -> índice
embedding_matrix = np.zeros((vocab_size, embedding_dim))

# Poblar matriz: palabras conocidas usan pre-entrenados,
# palabras desconocidas inicializan aleatoriamente
for idx, word in enumerate(word_list[:vocab_size]):
    word_index[word] = idx
    if word in w2v:
        # Palabra conocida: copiar embedding pre-entrenado
        embedding_matrix[idx] = w2v[word]
    else:
        # Palabra desconocida: inicializar de N(0, 1)
        embedding_matrix[idx] = np.random.normal(
```

Pipeline Completo con Embeddings Promediados

```python
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np

# ============================================
# 1. CARGA DE DATOS
# ============================================
(x_train, y_train), (x_test, y_test) = \
    keras.datasets.imdb.load_data(num_words=10000)

# IMDb devuelve secuencias de índices de palabras
# x_train[0] es una lista de enteros: [1, 14, 22, 16, ...]
print(f"Shape de entrada: {x_train[0].shape}")  # (variable)

# ============================================
# 2. CONVERSIÓN A EMBEDDINGS PROMEDIADOS
```

Visualización con t-SNE

Los embeddings viven en espacios de 300 dimensiones. Para visualizar, usamos **t-SNE** (t-Distributed Stochastic Neighbor Embedding), que preserva distancias locales.

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

# Reducir a 2D (puede tomar minutos)
tsne = TSNE(n_components=2, random_state=42,
            perplexity=30, n_iter=1000)
embeddings_2d = tsne.fit_transform(embedding_matrix[:2000])

# Graficar
fig, ax = plt.subplots(figsize=(14, 10))
ax.scatter(embeddings_2d[:, 0], embeddings_2d[:, 1],
           alpha=0.4, s=20, color='steelblue')

# Anotaciones para palabras seleccionadas
```

Función para Resolver Analogías

```
def resolver_analogia(w1, w2, w3, modelo, top_n=5):
    """
    Resuelve: w1 es a w2 como w3 es a ?

    Ejemplo: "rey" es a "hombre" como "reina" es a ?
    """
    try:
        resultados = modelo.most_similar(
            positive=[w2, w3],
            negative=[w1],
            topn=top_n
        )
        return resultados
    except KeyError as e:
        return f"Palabra desconocida: {e}"
```

Las Cuatro Ideas Centrales

1. **One-hot encoding es ineficiente**
   - Espacios enormes y dispersos
   - No captura similitud semántica
2. **Embeddings codifican significado en geometría**
   - Palabras similares → vectores cercanos
   - Relaciones semánticas → operaciones vectoriales
   - Compresión: 10k dims → 300 dims
3. **Distributional hypothesis como fundamento**
   - Contexto similar → palabras similares
   - Aprender de corpus no etiquetado (unsupervised)
4. **Transfer learning es estándar**
   - No entrenar embeddings desde cero
   - Usar pre-entrenados (Word2Vec, GloVe, FastText)
   - Fine-tune según necesidad

Por Qué Importan los Embeddings

- **Compacidad**: 300 dimensiones vs. 10,000+ dimensiones one-hot
- **Significado**: Capturan relaciones semánticas reales
- **Transferibilidad**: Úsalos en múltiples tareas
- **Eficiencia**: Pre-entrenados, listos para usar

Tabla de Troubleshooting

Problema	Causa Raíz	Solución
Palabras OOV	Vocabulario pre-entrenado no cubre palabra rara	Usar FastText (subpalabras) o reentrenar
Baja exactitud	Promediado pierde orden y contexto	Usar RNN/LSTM (próxima clase)
Archivos grandes	Embeddings completos pesan GB	Usar dimensiones menores (50-100)
Overfitting	Dataset pequeño + modelo complejo	Congelar embeddings (`trainable=False`)
Sin diferenciación	Todas las palabras se mapean igual	Revisar: ¿matriz inicializada correctamente?
Analogías falsas	Relaciones ruidosas en embeddings	Entrenados en corpus sesgado (revisar datos)

Actividades Hands-On

1. **Comparación de embeddings**
   - Carga Word2Vec y GloVe
   - Compara resultados en tareas de analogía
   - ¿Cuál es mejor para tu dominio?
2. **Estudio de dimensionalidad**
   - Entrena modelos con embeddings de 50, 100, 200, 300 dims
   - ¿Hay punto de saturación?
   - Trade-off: exactitud vs. velocidad
3. **Frozen vs. Trainable**
   - Modelo A: embeddings congelados (`trainable=False`)
   - Modelo B: embeddings ajustables (`trainable=True`)
   - ¿Cuál generaliza mejor en dataset pequeño?
4. **Caza de analogías**
   - Encuentra 10 analogías que funcionan bien
   - Encuentra 5 que fallan
   - Analiza por qué fallan
5. **Visualización t-SNE**
   - Visualiza 3000+ embeddings

Limitaciones Actuales

El promediado de embeddings funciona, pero ignora:

- **Orden de palabras**: “no bueno” vs. “bueno no” → mismo embedding
- **Contexto variable**: Mismo verbo con significados diferentes según contexto
- **Dependencias a largo plazo**: Relaciones entre palabras lejanas

Ejemplo de limitación:

Oración 1: “Este película **no es** buena” → promedio ≈ “negativo” Oración 2: “Esta película es **muy** buena” → promedio ≈ “positivo”

Cambio de una palabra invierte sentimiento, pero promediado es sensible.

Solución: Modelos Secuenciales

Clase 3: Clasificación de Texto con RNNs/LSTMs

- Procesar palabras **en orden**
- Recordar contexto acumulativo a través de secuencias
- Capturar dependencias a largo plazo
- Mejora esperada: 89% → 95%+

Papers Clásicos

- **Mikolov et al. (2013)**. “Efficient Estimation of Word Representations in Vector Space”. ICLR. [Word2Vec original]
- **Pennington, Socher & Manning (2014)**. “GloVe: Global Vectors for Word Representation”. EMNLP. [GloVe]
- **Firth, J. R. (1957)**. “A Synopsis of Linguistic Theory”. Studies in Linguistic Analysis. [Hipótesis distribucional]

Recursos Implementación

- **Gensim**: https://radimrehurek.com/gensim/
- **Spacy**: https://spacy.io/
- **TensorFlow Embedding Layer**:
  https://www.tensorflow.org/api_docs/python/tf/keras/layers/Embedding

Visualización y Análisis

- **TensorBoard**: Visualización interactiva de embeddings (projector.tensorflow.org)
- **t-SNE**: van der Maaten & Hinton (2008), “Visualizing Data using t-SNE”

Lecciones clave

Los embeddings transforman lo simbólico en geométrico.

Pasamos de:

- Palabras como etiquetas sin relación → **Palabras como puntos en un espacio semántico**
- Diccionarios discretos → **Espacios continuos con estructura**
- Vocabulario cerrado → **Capacidad de generalización a palabras nuevas** (especialmente FastText)

Este puente entre lenguaje y geometría es lo que permite a las redes neuronales razonar sobre significado.

Próximas Preguntas

¿Cómo capturamos **orden y contexto** dinámico?

¿Cómo manejamos **ambigüedad** (misma palabra, múltiples significados)?

¿Cómo escalamos a **idiomas morfológicamente complejos**?

→ **Respuestas en clases futuras con RNNs, Transformers y embeddings contextuales (ELMo, BERT)**

Preguntas y Discusión

Gracias por la atención.

Preguntas frecuentes: - ¿Puedo combinar múltiples esquemas de embeddings? - ¿Qué pasa si mi idioma no tiene embeddings pre-entrenados? - ¿Cómo detecto y mitigo sesgos en embeddings?

Siguiente: Deep dive en RNNs para procesamiento secuencial.